

Chapter 6

Interfaces: Families of Classes

In Chapter 4, we concluded that a good subclass relationship exists when the subclass embodies the same behavior as the parent superclass. Sometimes, multiple classes embody the same behavior, but there is no compelling reason to make any of them be a parent class. For instance, the three lists described in Chapter 5 are not built on each other in any way; they are just different concrete implementations of the same specified behavior.

An *interface* specifies behavior in the abstract. It is a bundle of methods devoid of implementation. It is a template for a contract; a class that claims to implement an interface is signing a contract that states that it provides the behavior specified by that interface. In Java, all interface methods are necessarily `public` instance methods, so no access modifier need be stated. Method bodies are not given. Instead, a semicolon appears where the braced block would normally be. Since Java 8, methods can be defined with a default implementation, so long as it can be written in terms of methods known to be available.

This chapter revisits the nodes of the zeroless random-access list, separating the nodes that hold just a single value from those that hold two values at the type level, to avoid even the possibility of accessing inappropriate fields, using a common interface to allow interoperation. We also declare the ordinary linked list and the zeroless random-access list as implementors of a common interface, which we shall call a `Polist` (for “prefix-oriented list”, to distinguish from other potential interfaces). This `Polist` is as follows:

<lists-redux/Polist.java 67>≡

```
public interface Polist<E> {  
    int size();  
    void prepend(E x);  
    E drop();  
    E get();  
    E get(int index);  
}
```

67

6.1 Revised Zeroless Random-Access List

The nodes of the zeroless random-access list come in two forms: one which represents a bit with value 1 (containing one value), and one which represents a bit with value 2 (containing two values). The original `Node` class, parameterized by a type `T`, defines the following helper methods, types slightly adjusted:

```
68a <ZRAL node interface 68a>≡ (70)
    private static interface Node<T> {
        int size(); // get the size of the list
        Node<T> prepend(T x); // insert x at the beginning
        Pair<? extends Node<T>,T> drop(); // remove and return a value
        T get(int index); // return (but retain) first value
    }
```

We shall say that a node which holds a single value will be called a `OneNode`. This class is responsible only for nodes of this kind, though it does create others. We state that it implements this interface by using the “implements” keyword.

```
68b <ZRAL OneNode 68b>≡ (70)
    private static class OneNode<T> implements Node<T> {
        private final T data;
        private final Node<Pair<T,T> > rest;
        public OneNode(T x, Node<Pair<T,T> > list) {
            data = x; rest = list;
        }
        @Override public int size() {
            if (rest == null) { return 1; }
            return 1 + 2*rest.size();
        }
        @Override public TwoNode<T> prepend(T x) {
            return new TwoNode<>(x, data, rest);
        }
        @Override public Pair<TwoNode<T>,T> drop() {
            if (rest == null) { return new Pair<>(null, data); }
            Pair<? extends Node<Pair<T, T> >,Pair<T,T> >
                p = rest.drop();
            T x = p.second.first; T y = p.second.second;
            return new Pair<>(new TwoNode<>(x, y, p.first), data);
        }
        @Override public T get(int index) {
            if (index == 0) { return data; }
            if (rest == null) {throw new IndexOutOfBoundsException();}
            int i = index - 1;
            if (i%2 == 0) { return rest.get(i/2).first; }
            return rest.get(i/2).second;
        }
    }
```

The nodes with two values are defined similarly. A node with one value has a simple prepend (increment size) but may borrow on drop (decrement size); a node with two values may carry on prepend but has a simple drop. The constructor, size query, and indexed access must take into account the additional value, but are otherwise nearly identical in structure to that of a `OneNode`.

```

⟨ZRAL TwoNode 69⟩≡
private static class TwoNode<T> implements Node<T> {
    private final T first;
    private final T second;
    private final Node<Pair<T,T> > rest;
    public TwoNode(T x, T y, Node<Pair<T,T> > list) {
        first = x; second = y; rest = list;
    }
    @Override public int size() {
        if (rest == null) { return 2; }
        return 2 + 2*rest.size();
    }
    @Override public OneNode<T> prepend(T x) {
        Pair<T,T> c = new Pair<>(first, second);
        if (rest == null) {
            return new OneNode<>(x, new OneNode<>(c, null));
        }
        return new OneNode<>(x, rest.prepend(c));
    }
    @Override public Pair<OneNode<T>,T> drop() {
        return new Pair<>(new OneNode<>(second, rest), first);
    }
    @Override public T get(int index) {
        if (index == 0) { return first; }
        if (index == 1) { return second; }
        if (rest == null) {throw new IndexOutOfBoundsException();}
        int i = index - 2;
        if (i%2 == 0) { return rest.get(i/2).first; }
        return rest.get(i/2).second;
    }
}

```

(70) 69

Notice that the prepend and drop operations for each kind of node return nodes of the opposite kind. Never is it necessary for a node to question what kind of node it is, nor what kinds surround it. Nodes of a particular type can be constructed explicitly, but everything else can just go through the common interface. With the node classes defined, we can build up the wrapper class.

```

70 <lists-redux/ZerolessRandomAccessList.java 70>≡
    import java.util.NoSuchElementException;
    public class ZerolessRandomAccessList<E> implements Polist<E> {
        private Node<E> head;
        public ZerolessRandomAccessList() { head = null; }

        @Override
        public int size() {
            if (head == null) { return 0; }
            return head.size();
        }

        @Override
        public void prepend(E x) {
            if (head == null) { head = new OneNode<>(x, null); }
            else
                { head = head.prepend(x); }
        }

        @Override
        public E drop() {
            if (head == null) {throw new NoSuchElementException();}
            Pair<? extends Node<E>,E> result = head.drop();
            head = result.first;
            return result.second;
        }

        @Override
        public E get() { return get(0); }

        @Override
        public E get(int index) {
            if (head == null) {throw new IndexOutOfBoundsException();}
            return head.get(index);
        }

        <ZRAL node interface 68a>
        <ZRAL OneNode 68b>
        <ZRAL TwoNode 69>
        <random-access list pair 59b>
    }

```

6.2 Ordinary Linked Lists

The `LinkedList` class is essentially unchanged from Chapter 5. It need only state that it implements the `Polist` interface.

```

<lists-redux/LinkedList.java 71>≡
import java.util.NoSuchElementException;
public class LinkedList<E> implements Polist<E> {
    private Node<E> head;
    public LinkedList() { head = null; }

    @Override
    public int size() {
        if (head == null) { return 0; }
        return head.size();
    }

    @Override
    public void prepend(E x) {
        head = new Node<>(x, head);
    }

    @Override
    public E drop() {
        if (head == null) {throw new NoSuchElementException();}
        E out = head.data;
        head = head.rest;
        return out;
    }

    @Override
    public E get() { return get(0); }

    @Override
    public E get(int index) {
        if (head == null) {throw new IndexOutOfBoundsException();}
        return head.get(index);
    }
    <singly linked list internal node 58>
}

```

Figure 6.1 represents a portion of the classes that we have described so far as a UML class diagram. The classes `ZerolessRandomAccessList`, `LinkedList`, `OneNode`, and `TwoNode` are shown in simplified form, without their fields or methods. The diagram shows two ways to show that a class implements an interface: one way is to use an arrow notation like a dashed version of that used to represent a subclass relationship, and the other is a ball-and-socket notation. In the former, the arrow is drawn from the implementor to the implemented interface. In the latter, a ball labeled by the name of an interface is used to indicate implementation of that interface, while a socket is used to indicate use of that interface. The notation “ $\llbracket \text{bind} \rrbracket P \rightarrow V$ ” means that the implementation or use of

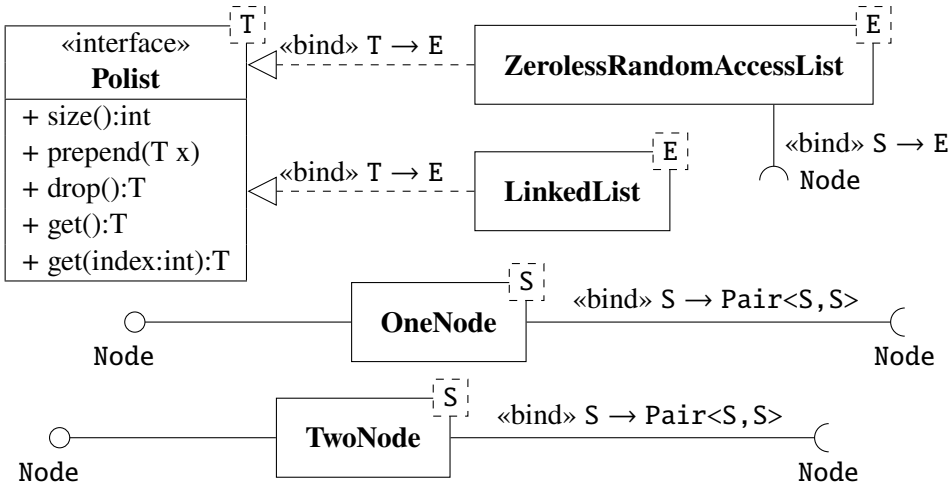


Figure 6.1: A UML class diagram for some classes and interfaces defining our lists.

the connected component fills in the parameter P with the value V . For instance, a `ZerolessRandomAccessList` uses a `Node` whose type parameter (here “S”) is the same as that of the list itself (here “E”). Each of the `OneNode` and `TwoNode` classes implements the interface called `Node` and also uses another `Node`, where the latter holds a pair instead of a single value. The ball-and-socket notation makes visually apparent that the `OneNode` and `TwoNode` can connect to one another as a chain. As the `Node` interface of the `ZerolessRandomAccessList` has the same name as the `Node` class of the `LinkedList`, a diagram that included both would need to disambiguate them in some way, perhaps including the parent class name.

With this structure, a method can ask for a `Polist<? super Square>` as an argument, and it will have no idea whether the list that it receives is a `LinkedList<Rectangle>`, a `ZerolessRandomAccessList<Square>`, or even something else entirely. All it knows is that it now has a list into which it can prepend squares. The implementation of a method defined this way cannot be not tightly coupled to the concrete structure of the data; it accesses only through the methods exposed by the contract of the interface.

6.3 Stacks

Like a list, a *stack* represents a collection of data. Items can be inserted into the stack (pushed) or removed from the stack (popped), where a removal operation retrieves the element in the structure that was inserted most recently. This is also called a *last-in first-out* structure, *LIFO* for short. Typically there is an operation called “top” or “peek” to view the item that would next be removed, without actually

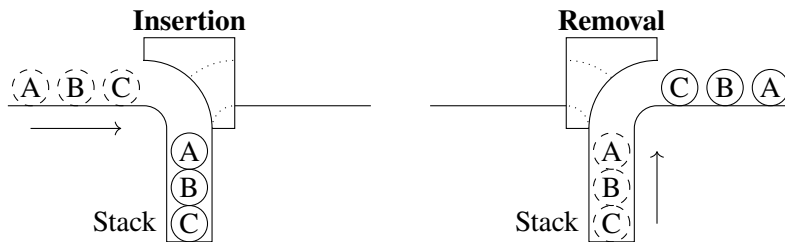


Figure 6.2: Pushing three items into a stack and then popping them back out.

removing it; this can be implemented by removing the value, then pushing it back while retaining a reference to it, so this is not an essential operation. As the removal and peek operations are valid only when the stack is nonempty, typically there is an operation available that can be used to check this condition. That might be a boolean value like “isEmpty()” or its complement “hasContent()”, or it might be a “size()” query. A sufficient interface to model this kind of structure is as follows.

<the stack interface 73>≡

```
public interface Stack<E> {
    void push(E x);
    E pop();
    E top();
    boolean isEmpty();
}
```

73

A stack is so-named because its operation resembles that of a physical stack of plates, like the spring-loaded plates in a cafeteria. When new, clean plates are pushed onto the stack from above, older plates are pushed down into the well. The plate added most recently is on top, and this is where they are pulled from. When a plate is removed from the top of the stack, the one beneath it is pushed up into position and becomes the new top. Figure 6.2 demonstrates what happens if three items are inserted into a stack, then afterward all three are removed. Notice that the output comes in reverse order. First, C is inserted, then B atop it, then A on that. Then A is removed, B follows, and C comes out last.

You may have heard of programs receiving a *stack overflow* error when executing deeply nested function calls. Function evaluation can be modeled using a stack, usually an indexable stack, so that you can refer to items other than the value inserted most recently. To call a function, its arguments are placed, in order, on the stack, followed by the location to which to return when execution finishes. The function itself places any local variables that it defines into this stack as well. In Java, the call-stack holds only values of primitive type and references to values of reference type, but in other languages like C++ you might find large data structures embedded in the call-stack as well. The return location, parameters, and local

variables for a particular instance of a function call are called its *stack frame*. Stack frames are pushed to the stack upon call and popped from the stack upon return.

Any `Polist` can be modified to implement the stack interface. To `push(x)` is to `prepend(x)`. To `pop()` is to `drop()`. The `top()` is the item retrieved with `get()`. And the list `isEmpty()` if its `size()` is zero. Consulting Table 5.2, this means that for a `LinkedList`, all operations are constant-time except perhaps `isEmpty` (but that can be made constant-time with trivial modification), and for a `ZerolessRandomAccessList`, the push and pop operations require logarithmic time, while the top can be viewed in constant time and, again with trivial modification, emptiness can also be queried in constant time.

6.4 Queues

A *queue* also represents a collection of data into which items can be inserted and from which items can be retrieved. While a stack operates in last-in first-out order, a queue operates in *first-in first-out* order, so it is sometimes called a *FIFO*. Imagine lining up to enter a concert venue; the first party at the door is the first party let in.

For a queue, the insertion operation is called “enqueue” and the removal operation is called “dequeue”. Unlike in the case of a stack, one cannot peek at the next available value by means of removal and reinsertion, as this would send the inspected element to the back of a queue rather than replacing it in the front. A minimal interface is as follows:

```
74 <the queue interface 74>≡
    public interface Queue<E> {
        void enqueue(E x);
        E dequeue();
        boolean isEmpty();
    }
```

Given two stacks, they can be arranged into an inbox–outbox system to form a queue. To enqueue an item, simply push it into the inbox. The queue is empty if both boxes are empty. To dequeue an item from a nonempty queue, first check if the outbox has content. If so, pop an item from there. Otherwise, flush the inbox into the outbox by popping one element from the inbox and pushing it into the outbox, then repeating this process until the inbox is empty. Pushing a series of elements into the inbox means that they will be popped in reverse order; pushing this reversed series of elements into the outbox means that they will be popped in the original order. This maintains the desired first-in first-out property of the queue.

Suppose that we are using a `LinkedList` for the stack, modified such that all relevant operations take constant time. Enqueueing elements is always a constant-time operation, as that is mere stack insertion. However, dequeueing elements is only fast if the outbox is nonempty. After n insertions with an empty outbox, the

queue is of size n , so a dequeue operation will require at least n pops to remove all elements from the inbox and $n - 1$ pushes to insert all but the last into the outbox (as the last is the item to return, there is no need to insert it). In other words, $2n - 1$ constant-time operations are performed, so an individual dequeue operation runs in linear time, $\Theta(n)$. However, the next $n - 1$ dequeue operations are then guaranteed to run in constant time! So to run n dequeue operations seems like it should take linear time, and each individual operation should therefore be essentially constant-time.

6.5 Amortized Complexity Analysis

Using *amortized analysis*, modeling the cost of operations in sequence rather than in isolation, we can argue that dequeuing requires essentially constant time. One model of amortized analysis is the *banker's model*, in which each item in the structure maintains a conceptual bank account. Every operation has a true cost associated with it. We can say that insertion into and removal from a `LinkedList`-based stack each cost one credit. An item's account may never go negative.

Let us apply this concept to an inbox–outbox system using two `LinkedList`-based stacks as a queue. We can say that enqueueing an item costs three credits. One of these is used to pay for the insertion into the inbox, while the remaining two are saved in that item's account. Every item in the inbox has two credits at all times. When a dequeue is requested with an empty outbox, each item in the inbox pays one credit to leave the inbox, then another credit to be inserted into the outbox. The account of each item in the outbox is left empty. By paying in one credit, an item can be removed from the outbox. So dequeuing costs one credit. The heightened cost of insertion pays for the later removal. Both of these operations are charged a constant amount of work: enqueueing is a three-credit affair and dequeuing a one-credit affair. So we can say that each is amortized constant-time. Sometimes you will see this written “amortized $\Theta(1)$ ”; I prefer the compact notation $\hat{\Theta}(1)$.

The analysis for a `ZerolessRandomAccessList` is similar, except that the basic stack operations cost $\log n$, where n is one more than the size of the list. Here, we can say that enqueueing costs three times the logarithm of the size of the inbox. One set is used to enter the inbox. One set is used to exit it later. And the last is used to pay for the insertion of its counterpart: the item that will end up in the same position in the outbox as that item occupied in the inbox. That is, if the inbox held items A, B, C, D, and E, then A pays for E's insertion and vice versa, B pays for D's insertion and vice versa, and C pays for itself. This shows that, under amortized analysis, enqueueing and dequeuing are operate in $\hat{\Theta}(\log n)$ time when a queue is built over an inbox–outbox system using `ZerolessRandomAccessList`-based stacks. The isolated runtime of a single dequeue operation, however, could be $\Theta(\log n!)$. This is $\log 1 + \log 2 + \dots + \log n = \log(1 \cdot 2 \cdot \dots \cdot n) = \log n!$. Using

Table 6.1: Performance of stack and queue operations.

		LinkedList	ZerolessRandomAccessList
Stack	push	$\Theta(1)$	$\Theta(\log n)$
	pop	$\Theta(1)$	$\Theta(\log n)$
	top	$\Theta(1)$	$\Theta(1)$
Shared	isEmpty	$\Theta(1)$	$\Theta(1)$
Queue	enqueue	$\Theta(1), \hat{\Theta}(1)$	$\Theta(\log n), \hat{O}(\log n)$
	dequeue	$\Theta(n), \hat{\Theta}(1)$	$\Theta(n \log n), \hat{O}(\log n)$

Stirling's approximation, we see that this is the same as saying the operation is linearithmic, $\Theta(n \log n)$, in isolation.

Standard complexity analysis is useful when you want real-time performance for every single operation, such as when you have hard limits on resources. Amortized complexity analysis is useful when individual operations may sometimes have high cost but you are concerned with how the system behaves in the long run across a large series of operations. Table 6.1 lists some analyses of stack and queue operations for these two kinds of (persistent) list data structures, where queue operations assume the use of an inbox–outbox system. There are, of course, other ways to more directly implement a queue. Using an ephemeral linked list with some additional tracking instead of an inbox–outbox system, all stack and queue operations can be constant-time even under standard analysis.

6.6 Summary

An interface describes a set of behaviors that are contractually required to be present in classes that claim to implement that interface. They are used to create families of types with overlapping behaviors. Interfaces can be used instead of concrete types as restrictions on parameters, as return types, or in generics. An interface cannot itself be constructed; there is no such thing as a `Polist(...)` constructor even in presence of a `Polist` interface. Instead, some concrete type implementing the interface must be created. You might define a variable as follows:

```
Polist<String> inputs = new LinkedList<>();
```

When there is some abstract behavior that you care about, decoupled from any implementation details, such as the last-in first-out sequencing of a `Stack` or the first-in first-out sequencing of a `Queue`, it makes sense to define this behavior as an interface. Then you can define one or more concrete classes to implement these interfaces. It may be that different concrete implementations have different

performance characteristics, so you may find that one implementation is better in one part of your application, while another is better in another part. Methods written in terms of the interface are decoupled from the implementation details, so this mixed use is possible without having to write additional support code.

In redefining the zeroless random-access list, using an interface let us separate nodes that hold a single value from nodes that hold two values at the type level. The return types of the individual implementing classes can be set to enforce correct behavior, such as the fact that incrementing a `OneNode` always results in a `TwoNode` and vice versa. This protects against writing erroneous code by ensuring that you do not accidentally create the wrong kind of value. The stronger guarantees at the type level reduce opportunity to inspect invalid fields or otherwise return incorrect results.

The performance characteristics of a system depend not on the interface but on the concrete implementation. If you care about the timings of individual operations, then standard asymptotic analysis is the way to go. This tells you if there might be spikes in latency to perform some operations, like to dequeue from a queue defined by an inbox–outbox system. Sometimes, however, you care not about the speed of an individual operation, but a batch of operations taken as a whole. In this case you would use amortized analysis, and under this framework we can say that dequeuing from an inbox–outbox system is, essentially, constant time when the stacks are built on plain linked lists, or essentially logarithmic when the stacks are built on random-access lists. The stack operations on the latter can also be amortized; I suggest running this analysis to see what, if anything, changes!